

StockFlow

Gestion de stock et industrialisation DevOps

Architecture, implémentation et chaîne d'intégration continue

Auteurs : Ghorbel Elyes

Souissi Mohamed Aziz

Spécialité : Ingénierie Cloud et Virtualisation

Établissement : iTeam University

Dépôt : [https://github.com/elyescesar/
pfa-gestion-stock-cicd](https://github.com/elyescesar/pfa-gestion-stock-cicd)

Table des matières

1	Introduction	1
1.1	Contexte et motivation	1
1.2	Périmètre du projet	1
1.3	Organisation du dépôt	1
1.4	Modes d'exécution	1
1.5	Organisation du rapport	2
1.6	Conventions documentées	2
2	Analyse du besoin et objectifs	3
2.1	Problématique métier	3
2.2	Acteurs et rôles	3
2.3	Exigences fonctionnelles	3
2.4	Exigences non fonctionnelles	3
2.5	Objectifs techniques	3
2.6	Hors périmètre	5
3	Architecture globale	6
3.1	Vue logique	6
3.2	Deux topologies de déploiement	6
3.2.1	Mode développement (Docker Compose)	6
3.2.2	Mode validation Kubernetes (minikube)	6
3.2.3	Production optionnelle (VPS k3s)	7
3.3	Flux de données principaux	7
3.3.1	Authentification	7
3.3.2	Mouvement de stock	7
3.3.3	Observabilité	7
3.4	Chaîne d'outillage DevOps	7
3.5	Choix architecturaux structurants	7
4	Backend — API FastAPI	9
4.1	Stack et organisation du code	9
4.2	Configuration	9
4.3	Modèle de domaine	9
4.3.1	Entités	9
4.3.2	Diagramme entité-association	10
4.4	Sécurité et authentification	10
4.5	Logique métier des mouvements	10
4.6	Surface API REST	11
4.7	Initialisation des données	11
4.8	Migrations Alembic	11
4.9	Conteneurisation de l'API	11

5	Frontend — interface StockFlow	12
5.1	Stack technique	12
5.2	Architecture frontale	12
5.2.1	Organisation des sources	12
5.2.2	Routage et protection	12
5.3	Client API	12
5.4	Pages fonctionnelles	13
5.4.1	Connexion (<code>PageConnexion.tsx</code>)	13
5.4.2	Tableau de bord	13
5.4.3	Produits	13
5.4.4	Catégories	13
5.4.5	Mouvements	13
5.4.6	Alertes	13
5.5	Composants UI	13
5.6	État global	13
5.7	Layout responsive	14
5.8	Build et livraison	14
5.9	Parcours utilisateur et états d'interface	14
5.10	Tests frontend	14
6	Persistance et modèle de données	15
6.1	PostgreSQL comme source de vérité	15
6.2	Schéma détaillé	15
6.2.1	Table <code>categorie</code>	15
6.2.2	Table <code>utilisateur</code>	15
6.2.3	Table <code>produit</code>	15
6.2.4	Table <code>mouvement</code>	15
6.3	Requêtes métier typiques	16
6.3.1	Alertes	16
6.3.2	Tableau de bord	16
6.4	Stratégie de démarrage du schéma	16
6.5	Données de démonstration	16
6.6	Flux transactionnel d'un mouvement	16
6.7	Relations et cardinalités	17
6.8	Index et performances	17
6.9	Comparaison SQLite (tests) et PostgreSQL (runtime)	17
6.10	Cohérence et limites	17
7	Conteneurisation Docker	18
7.1	Rôle de Docker dans le projet	18
7.2	Docker Compose — développement	18
7.2.1	Services	18
7.2.2	Variables d'environnement API	18
7.2.3	Script de démarrage	18
7.3	Dockerfile backend	18
7.4	Dockerfile frontend	19
7.5	Images et CI	19
7.6	Réseau et volumes	19
7.7	Bonnes pratiques	19

8	Kubernetes et Helm	20
8.1	Kubernetes avec minikube	20
8.1.1	Principe	20
8.1.2	Démarrage	20
8.1.3	Production optionnelle (VPS k3s)	20
8.2	Namespaces (minikube)	20
8.3	Chart Helm <code>pfa-stock</code>	20
8.3.1	Values	20
8.3.2	Ressources déployées	21
8.3.3	Ingress et routage (minikube)	21
8.4	Observabilité et outils sur le cluster	21
8.5	Cycle de validation	21
9	Infrastructure as Code — Terraform	22
9.1	Objectifs de Terraform dans StockFlow	22
9.2	Providers	22
9.3	Variables	22
9.4	Ressources par fichier	22
9.4.1	<code>kubernetes.tf</code>	22
9.4.2	<code>cert-manager.tf</code>	22
9.4.3	<code>helm.tf</code>	22
9.4.4	<code>grafana_dashboard.tf</code>	23
9.5	Exécution opérationnelle	23
9.5.1	Validation (CI et local)	23
9.5.2	Apply production (optionnel)	23
9.6	Terraform vs scripts minikube	23
10	Observabilité — Prometheus et Grafana	24
10.1	Objectifs	24
10.2	Instrumentation applicative	24
10.3	kube-prometheus-stack	24
10.3.1	Démo locale (minikube)	24
10.3.2	Production optionnelle (VPS)	24
10.4	ServiceMonitor	25
10.5	Flux de métriques	25
10.6	Dashboard Grafana	25
10.7	Accès opérationnel	25
10.7.1	Démo soutenance (minikube)	25
10.7.2	Production optionnelle	25
10.8	Mode développement Compose	26
10.9	Alerting	26
11	Intégration continue et déploiement — GitHub Actions	27
11.1	Principes	27
11.2	Déclencheurs	27
11.3	Pipeline CI	27
11.4	Pipeline CD optionnel (<code>deploy.yml</code>)	27
11.5	Comparaison CI vs local	27
11.6	Gouvernance	27

12 Sécurité, tests et qualité	29
12.1 Modèle de menace (périmètre local)	29
12.2 Authentification et autorisation	29
12.3 Secrets et configuration	29
12.4 CORS	29
12.5 Réseau Kubernetes	29
12.6 Tests automatisés	30
12.6.1 Backend (pytest)	30
12.6.2 Frontend (Vitest)	30
12.6.3 Smoke tests (<code>make verify</code>)	30
12.7 Analyse statique	30
12.8 Pistes de durcissement production	30
13 Exploitation et scripts d'automatisation	31
13.1 Makefile comme point d'entrée	31
13.2 Scripts shell	31
13.3 Guide soutenance	32
13.4 URLs	32
13.4.1 Développement local (Compose)	32
13.4.2 Validation Kubernetes (minikube)	32
13.4.3 Production optionnelle (VPS k3s)	32
13.5 Dépannage	32
13.6 Logs	32
14 Conclusion et perspectives	33
14.1 Bilan	33
14.2 Objectifs atteints	33
14.3 Limites connues	33
14.4 Perspectives	33
14.5 Mot de fin	34
A Annexe A — Référence API REST	35
A.1 Auth	35
A.2 Catégories	35
A.3 Produits	35
A.4 Mouvements	35
A.5 Tableau de bord	35
A.6 Exports	36
A.7 Santé et métriques	36
A.8 Exemples de payloads JSON	36
A.8.1 Connexion	36
A.8.2 Création de produit	36
A.8.3 Mouvement de sortie	36
A.8.4 Tableau de bord (extrait de réponse)	36
A.9 Codes d'erreur HTTP courants	37
A.10 Exemples cURL (mode Compose)	37
B Annexe B — Configuration et versions	38
B.1 Variables d'environnement (<code>.env.example</code>)	38
B.2 Secrets GitHub Actions (production optionnelle)	38
B.3 Versions d'outils (référence)	38
B.4 Fichiers de référence rapide	38

Table des figures

3.1	Architecture logique de StockFlow	6
3.2	Déploiement minikube (vue simplifiée)	7
3.3	Chaîne d'outillage (validation CI + démo minikube)	7
4.1	Modèle de données simplifié	10
6.1	Séquence simplifiée — enregistrement d'un mouvement	16
10.1	Chemin des métriques	25

Liste des tableaux

1.1	Structure logique du dépôt	2
2.1	Acteurs et permissions	3
2.2	Exigences fonctionnelles principales	4
2.3	Exigences non fonctionnelles	4
3.1	Décisions architecturales et justification	8
4.1	Tables relationnelles	9
4.2	Résumé des endpoints	11
5.1	Structure frontend/src/	12
5.2	Pages et endpoints consommés	14
7.1	Services Compose (développement)	18
8.1	Ressources du chart pfa-stock	21
9.1	Releases Helm gérées par Terraform (VPS optionnel)	23
9.2	Terraform vs bootstrap minikube	23
11.1	CI vs commandes locales	28
12.1	Gestion des secrets	29
13.1	Commandes Make principales	31
13.2	Scripts du répertoire scripts/	31
A.1	Endpoints d'authentification	35
A.2	Erreurs API typiques	37
B.1	Variables d'environnement principales	38
B.2	Secrets repository pour deploy.yml (déclenchement manuel)	38
B.3	Versions d'outils (référence)	39

Chapitre 1

Introduction

1.1 Contexte et motivation

La gestion des stocks reste un besoin récurrent dans les PME, ateliers et entrepôts de taille modeste : suivre les quantités disponibles, enregistrer les entrées et sorties, anticiper les ruptures et disposer d’une vue consolidée pour la décision. Les tableurs partagés atteignent rapidement leurs limites (concurrence d’édition, absence d’API, traçabilité faible, déploiement non industrialisé).

StockFlow répond à ce besoin par une **application web full-stack** couplée à une **chaîne DevOps reproductible** : le même dépôt source permet de développer localement (Docker Compose + Adminer), de valider automatiquement le code (GitHub Actions, incluant un déploiement éphémère sur **minikube**) et de démontrer l’orchestration Kubernetes (Helm, Prometheus, Grafana, dashboard) via `make cluster`.

1.2 Périmètre du projet

Le périmètre fonctionnel couvre :

- authentification par jeton JWT avec rôles **admin** et **opérateur** ;
- gestion des catégories et des produits (SKU, quantité, seuil d’alerte) ;
- mouvements de stock (**entree** / **sortie**) avec contrôle de stock négatif ;
- tableau de bord synthétique et liste des produits sous le seuil.

Le périmètre technique couvre :

- API REST FastAPI et interface React (Vite, TypeScript, Tailwind) ;
- persistance PostgreSQL 16 ;
- conteneurisation Docker et orchestration Docker Compose pour le développement (Adminer pour la BDD) ;
- validation Kubernetes via **minikube** local et job CI (`values-minikube.yaml`) ;
- chart Helm **pfa-stock** et Terraform (validate CI ; apply prod optionnel) ;
- stack **kube-prometheus-stack** avec dashboard Grafana versionné ;
- pipelines GitHub Actions : CI (tests, minikube, validation IaC).

1.3 Organisation du dépôt

Le dépôt suit une séparation nette des responsabilités (tableau [1.1](#)).

1.4 Modes d’exécution

Deux modes coexistent :

TABLE 1.1 – Structure logique du dépôt

Répertoire	Rôle
<code>backend/</code>	API FastAPI, modèles SQLAlchemy, Alembic, tests py-test
<code>frontend/</code>	SPA React, build Vite, tests Vitest
<code>docker/</code>	Compose développement local
<code>helm/pfa-stock/</code>	Chart Kubernetes de l'application
<code>infrastructure/terraform/</code>	Plateforme K8s (ingress, TLS, monitoring)
<code>monitoring/</code>	Valeurs Prometheus/Grafana et dashboard JSON
<code>scripts/</code>	Automatisation démarrage et vérification
<code>.github/workflows/</code>	CI et déploiement production
<code>docs/demo-soutenance.md</code>	Guide validation soutenance (Compose + minikube)
<code>docs/vps-setup.md</code>	Production VPS optionnelle (annexe)

make dev Stack Docker Compose : frontend sur le port 3000, API sur le port 8000, Adminer sur le port 8080.

make cluster Cluster **minikube** local : chart Helm, kube-prometheus-stack, dashboard Kubernetes (voir `docs/demo-soutenance.md`).

CI GitHub Actions Six jobs dont **kubernetes** (minikube éphémère + smoke test `/health`).

Un déploiement production sur VPS (`deploy.yml`, `docs/vps-setup.md`) reste documenté comme **perspective optionnelle**, hors périmètre de validation devant le jury.

1.5 Organisation du rapport

Le chapitre 2 formalise les objectifs et les acteurs. Le chapitre 3 présente l'architecture globale et les flux principaux. Les chapitres 4 à 6 détaillent l'application ; les chapitres 7 à 9 l'infrastructure ; les chapitres 10 et 11 l'observabilité et l'intégration continue. Le chapitre 13 documente l'exploitation quotidienne. Les annexes recensent l'API REST et les variables de configuration.

1.6 Conventions documentées

- Les chemins de fichiers sont relatifs à la racine du dépôt sauf mention contraire.
- Les identifiants de démonstration (`admin@stock.tn`, etc.) sont documentés à titre opérationnel ; en production ils doivent être remplacés et stockés dans un gestionnaire de secrets (GitHub Secrets, Secrets Kubernetes).

Chapitre 2

Analyse du besoin et objectifs

2.1 Problématique métier

Un gestionnaire de stock doit pouvoir répondre rapidement aux questions suivantes :

1. Quels produits sont en rupture ou proches du seuil minimum ?
2. Quelles entrées et sorties ont eu lieu, par qui, et pour quel motif ?
3. Comment organiser le catalogue par familles (catégories) ?

Ces besoins imposent une base relationnelle cohérente, des règles métier transactionnelles (un mouvement de sortie ne doit pas rendre le stock négatif) et une interface qui reflète l'état en temps quasi réel après chaque opération.

2.2 Acteurs et rôles

TABLE 2.1 – Acteurs et permissions

Rôle	Compte démo	Capacités
Administrateur	admin@stock.tn	CRUD complet, suppression catégories/produits
Opérateur	opérateur@stock.tn	Consultation, création mouvements et entités (sauf suppressions réservées admin)

L'authentification repose sur un jeton JWT transmis dans l'en-tête **Authorization: Bearer**.
Le backend résout l'utilisateur courant à chaque requête protégée via la dépendance `obtenir_utilisateur_com`

2.3 Exigences fonctionnelles

2.4 Exigences non fonctionnelles

2.5 Objectifs techniques

Les objectifs techniques déclinés dans l'implémentation sont :

1. **Séparation des couches** : frontend statique, API stateless, base PostgreSQL.
2. **Deux chemins de démonstration** : Compose local (`make dev`) et minikube (`make cluster`) avec le même chart Helm qu'en CI.
3. **Automatisation** : `make demo` en local ; CI sur chaque push/PR.

TABLE 2.2 – Exigences fonctionnelles principales

ID	Description
F1	Connexion par email et mot de passe, émission d'un jeton d'accès.
F2	CRUD catégories (nom, description optionnelle).
F3	CRUD produits (nom, référence SKU unique, quantité, seuil d'alerte, catégorie).
F4	Enregistrement de mouvements entree ou sortie avec mise à jour atomique du stock.
F5	Rejet d'une sortie si le stock deviendrait négatif (stock_insuffisant).
F6	Tableau de bord : compteurs produits, catégories, mouvements, liste des alertes.
F7	Page alertes : produits dont quantite $\leq$ seuil_alerte .

TABLE 2.3 – Exigences non fonctionnelles

ID	Description
NF1	Conteneurisation : exécution reproductible via Docker.
NF2	Orchestration : déploiement sur Kubernetes avec Ingress, sondes de santé, ServiceMonitor.
NF3	IaC : infrastructure déclarative (Terraform + Helm).
NF4	Observabilité : métriques Prometheus sur /metrics , dashboard Grafana.
NF5	CI/CD : validation automatique et déploiement production (GitHub Actions).
NF6	Sécurité : mots de passe hashés (bcrypt), secrets Kubernetes pour données sensibles.
NF7	Documentation opérationnelle : README, Makefile, scripts make demo .

2.6 Hors périmètre

Les éléments suivants sont explicitement hors périmètre de la version documentée :

- déploiement production VPS automatique (workflow `deploy.yml` manuel uniquement) ;
- haute disponibilité multi-nœuds, autoscaling horizontal, TLS terminé par certificats réels ;
- gestion fine des permissions au-delà de deux rôles ;
- application mobile native ou mode hors-ligne.

Chapitre 3

Architecture globale

3.1 Vue logique

L'architecture suit un modèle en couches classique, enrichi d'une couche d'observabilité. La figure 3.1 synthétise les composants et leurs interactions.

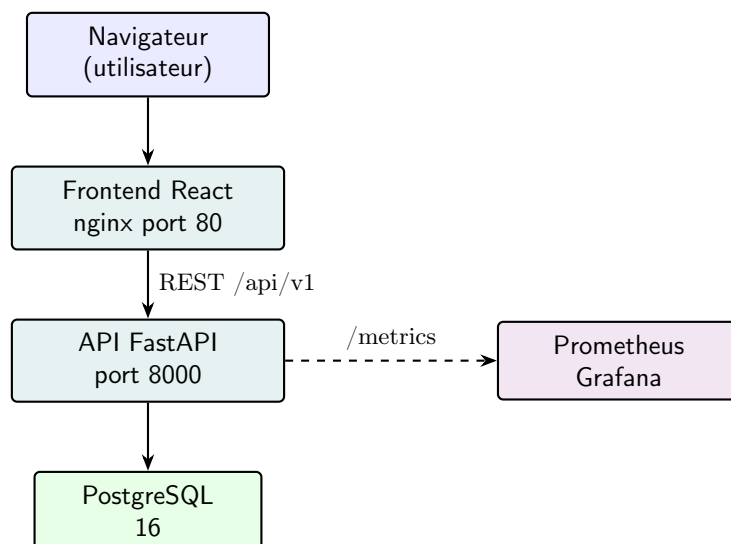


FIGURE 3.1 – Architecture logique de StockFlow

3.2 Deux topologies de déploiement

3.2.1 Mode développement (Docker Compose)

Le fichier `docker/compose.dev.yml` définit un réseau bridge `pfa` reliant `postgres`, `api`, `web` et `Adminer` (visualisation BDD, port 8080). Ports publiés : 3000 (UI), 8000 (API), 5432 (PostgreSQL), 8080 (Adminer).

3.2.2 Mode validation Kubernetes (minikube)

Le cluster `minikube` (profil `pfa`) est provisionné par `make cluster` : add-on `ingress`, `kube-prometheus-stack`, `pgAdmin`, dashboard Kubernetes, chart Helm `pfa-stock` (`values-minikube.yaml`). Les hôtes `pfa.test`, `api.pfa.test`, `grafana.test` et `pgadmin.test` sont mappés via `/etc/hosts` sur l'IP `minikube`.

3.2.3 Production optionnelle (VPS k3s)

Un déploiement sur VPS reste documenté (`docs/vps-setup.md`, workflow `deploy.yml` manuel) mais n'est pas requis pour la validation du PFA.

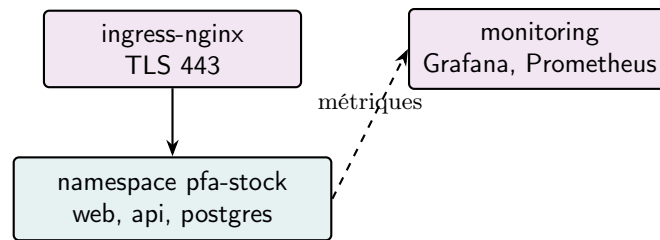


FIGURE 3.2 – Déploiement minikube (vue simplifiée)

Les scripts `demarrer-cluster-minikube.sh` installent la plateforme ; le job CI `kubernetes` reproduit le déploiement Helm sur un minikube éphémère.

3.3 Flux de données principaux

3.3.1 Authentification

1. Le client envoie `POST /api/v1/auth/connexion` avec email et mot de passe.
2. L'API vérifie le hash bcrypt, émet un JWT.
3. Le frontend stocke le jeton dans `sessionStorage`.

3.3.2 Mouvement de stock

Le service `gestion_stock.py` encapsule la logique transactionnelle (chapitre 4).

3.3.3 Observabilité

L'instrumentateur Prometheus expose `/metrics`. Sur minikube, Grafana est accessible sur `http://grafana.test` (après entrée `/etc/hosts`).

3.4 Chaîne d'outillage DevOps

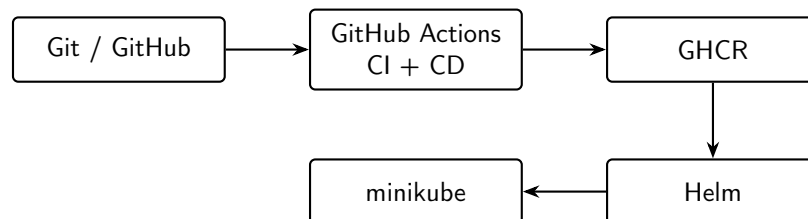


FIGURE 3.3 – Chaîne d'outillage (validation CI + démo minikube)

3.5 Choix architecturaux structurants

TABLE 3.1 – Décisions architecturales et justification

Décision	Justification
API stateless	Scaling horizontal des pods API ; session portée par JWT.
Frontend statique (nginx)	Surface d'attaque réduite en cluster.
PostgreSQL relationnel	Transactions ACID pour les mouvements.
minikube + Helm	Validation K8s locale et en CI sans VPS dédié.
GHCR + Helm (optionnel)	Images versionnées ; déploiement VPS manuel.
kube-prometheus-stack	Observabilité (Grafana, ServiceMonitor).

Chapitre 4

Backend — API FastAPI

4.1 Stack et organisation du code

Le backend est implémenté en **Python 3.12** avec **FastAPI 0.15**. Les dépendances principales figurent dans `backend/requirements.txt` : SQLAlchemy 2, Alembic, Pydantic Settings, `python-jose`, `passlib/bcrypt`, `prometheus-fastapi-instrumentator`.

L'arborescence `backend/app/` suit une séparation routes / schémas / modèles / services :

- `modeles/` — entités SQLAlchemy ;
- `schemas/` — modèles Pydantic entrée/sortie ;
- `routes/` — routeurs FastAPI par domaine ;
- `securite/` — JWT et hachage des mots de passe ;
- `services/` — logique métier (stock).

Le point d'entrée `main.py` instancie l'application, configure CORS, monte les routeurs sous le préfixe `/api/v1`, expose `/health` et instrumente `/metrics`.

4.2 Configuration

La classe `Parametres` (`config.py`) charge les variables d'environnement via `pydantic-settings` : `DATABASE_URL`, `SECRET_JWT`, `CORS_ORIGINS`, durée du jeton JWT, algorithme HS256.

En mode test (`MODE_TEST=1`), le cycle de vie de l'application ne crée pas les tables ni n'exécute le seed global : les tests utilisent une base SQLite en mémoire avec fixtures dédiées (`tests/conftest.py`).

4.3 Modèle de domaine

4.3.1 Entités

TABLE 4.1 – Tables relationnelles

Table	Clés	Attributs notables
<code>categorie</code>	PK <code>id</code>	<code>nom</code> unique, <code>description</code> nullable
<code>utilisateur</code>	PK <code>id</code>	<code>email</code> unique, <code>mot_de_passe_hash</code> , <code>role</code>
<code>produit</code>	PK, FK <code>id_categorie</code>	<code>reference_sku</code> unique, <code>quantite</code> , <code>seuil_alerte</code>
<code>mouvement</code>	PK, FK <code>produit</code> + <code>utilisateur</code>	<code>type_mouvement</code> , <code>quantite</code> , <code>motif</code> , <code>date_mouvement</code>

4.3.2 Diagramme entité-association

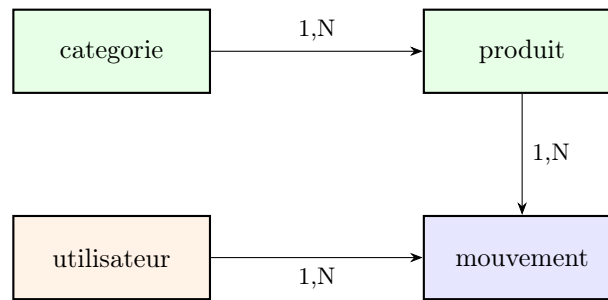


FIGURE 4.1 – Modèle de données simplifié

4.4 Sécurité et authentification

Les mots de passe ne sont jamais stockés en clair : `mots_de_passe.py` utilise `bcrypt`. Le module JWT (`securite/jwt.py`) :

Listing 4.1 – Émission et validation JWT (extrait)

```

1 def creer_jeton_acces(donnees: dict) -> str:
2     expiration = datetime.now(timezone.utc) + timedelta(
3         minutes=parametres.duree_token_minutes)
4     charge = donnees.copy()
5     charge.update({"exp": expiration})
6     return jwt.encode(charge, parametres.secret_jwt,
7                       algorithm=parametres.algorithme_jwt)

```

La dépendance `obtenir_utilisateur_courant` decode le jeton, extrait `sub` (email), charge l'utilisateur en base ou renvoie 401 `jeton_invalide`.

4.5 Logique métier des mouvements

Le cœur métier réside dans `services/gestion_stock.py`. L'algorithme :

1. Charger le produit par `id_produit` ; 404 si absent.
2. Calculer `nouvelle_quantite` : addition pour `entree`, soustraction pour `sortie`.
3. Si `nouvelle_quantite < 0`, lever HTTP 400 avec `detail=stock_insuffisant`.
4. Créer l'enregistrement `Mouvement`, mettre à jour `produit.quantite`, `commit`.

Listing 4.2 – Création d'un mouvement (extrait)

```

1 def creer_mouvement(session, donnees, id_utilisateur):
2     produit = session.query(Produit).filter(
3         Produit.id == donnees.id_produit).first()
4     if produit is None:
5         raise HTTPException(status_code=404, detail="
6             produit_introuvable")
7     nouvelle_quantite = calculer_quantite_apres_mouvement(
8         produit.quantite, donnees.type_mouvement, donnees.quantite)
9     if nouvelle_quantite < 0:
10         raise HTTPException(status_code=400, detail="stock_insuffisant
11             ")
12     # ... commit mouvement + mise a jour produit

```

4.6 Surface API REST

Tous les endpoints métier sont préfixés par `/api/v1`. Le tableau 4.2 résume les routes ; l'annexe A détaille les payloads.

TABLE 4.2 – Résumé des endpoints

Méthode	Chemins	Auth	Rôle
POST	<code>/auth/connexion</code>	Non	Émet JWT
GET	<code>/auth/moi</code>	Oui	Profil
CRUD	<code>/categories</code>	Oui	DELETE admin
CRUD	<code>/produits</code>	Oui	DELETE admin
GET	<code>/produits/alertes</code>	Oui	Sous seuil
GET/POST	<code>/mouvements</code>	Oui	Historique / création
GET	<code>/tableau-de-bord</code>	Oui	Agrégats
GET	<code>/health, /metrics</code>	Non	Sonde / Prometheus

4.7 Initialisation des données

`seed.py` peuple la base au premier démarrage (hors tests) : utilisateurs admin et opérateur, catégories et produits de démonstration. Cela permet une expérience immédiate après `make dev` sans script SQL manuel.

4.8 Migrations Alembic

Le répertoire `alembic/versions/` contient la migration initiale `001_initial.py` qui crée les quatre tables. `alembic/env.py` importe tous les modèles pour alimenter `target_metadata`. En exploitation courante, `Base.metadata.create_all` au démarrage assure la présence du schéma ; Alembic reste la référence versionnée pour audits et évolutions futures.

4.9 Conteneurisation de l'API

Le `backend/Dockerfile` utilise `python:3.12-slim`, installe `libpq-dev` pour `psycopg2`, copie `requirements.txt` puis le code, expose le port 8000 et lance :

```
1 uvicorn app.main:application --host 0.0.0.0 --port 8000
```

Les sondes Kubernetes (startup et liveness) ciblent `GET /health` sur le port 8000 (définies dans le template Helm `api.yaml`).

Chapitre 5

Frontend — interface StockFlow

5.1 Stack technique

L'interface utilisateur est une **single-page application** construite avec :

- **React 18** et **TypeScript 5.6** ;
- **Vite 6** comme bundler et serveur de développement ;
- **React Router 7** pour le routage côté client ;
- **Tailwind CSS 4** via le plugin `@tailwindcss/vite` ;
- **lucide-react** pour l'iconographie ;
- **Vitest** et **React Testing Library** pour les tests.

Le nom produit affiché est **StockFlow** : thème sombre, accents cyan, typographie Outfit (chargée depuis Google Fonts dans `index.html`).

5.2 Architecture frontale

5.2.1 Organisation des sources

TABLE 5.1 – Structure `frontend/src/`

Dossier	Contenu
<code>pages/</code>	Une page par route métier
<code>composants/Layout.tsx</code>	Shell navigation + zone <code>Outlet</code>
<code>composants/ui/</code>	Bibliothèque de composants réutilisables
<code>contexte/</code>	Auth et toasts globaux
<code>hooks/useFetch.ts</code>	Chargement données API avec état
<code>services/api.ts</code>	Client HTTP, gestion du jeton JWT

5.2.2 Routage et protection

`App.tsx` définit :

- `/connexion` — publique ;
- routes imbriquées sous `Layout` — protégées par `RoutesProtegees` qui lit `AuthContext` ;
- redirection vers `/connexion` si non authentifié ;
- affichage d'un chargeur pendant la vérification du jeton (`GET /auth/moi`).

5.3 Client API

`services/api.ts` centralise les appels :

- base URL depuis `import.meta.env.VITE_API_URL` (injectée au build Docker) ;
- jeton stocké dans `sessionStorage` sous la clé `jeton_acces` ;
- fonction `appelApi<T>()` ajoutant l'en-tête `Authorization` et gérant les erreurs 401 (déconnexion implicite).

En production, le build utilise `VITE_API_URL=https://api.pfa.elyes.dev` : les appels REST ciblent directement le sous-domaine API.

5.4 Pages fonctionnelles

5.4.1 Connexion (`PageConnexion.tsx`)

Formulaire email / mot de passe, valeurs préremplies en démo. Appel `POST /auth/connexion`, stockage du jeton, toast de succès, navigation vers le tableau de bord. Redirection automatique si session déjà valide.

5.4.2 Tableau de bord

Consomme `GET /api/v1/tableau-de-bord`. Affiche quatre cartes statistiques (produits, catégories, mouvements, alertes) et une liste des produits sous seuil avec barres de progression visuelles.

5.4.3 Produits

Liste filtrable (nom, SKU, catégorie), tableau avec badge d'alerte si stock bas, modal de création (`POST`), suppression avec confirmation (`DELETE`, visible au survol).

5.4.4 Catégories

Grille de cartes avec description tronquée, modal de création, suppression au survol.

5.4.5 Mouvements

Timeline verticale : entrées (vert) et sorties (rouge), date et motif. Modal pour enregistrer un mouvement (type, produit, quantité, motif optionnel).

5.4.6 Alertes

Cartes par produit en alerte avec jauge et pourcentage par rapport au seuil ; lien vers les mouvements pour corriger le stock.

5.5 Composants UI

La couche `composants/ui/` homogénéise l'interface :

- `Bouton` — variantes primaire, secondaire, danger, fantôme ; état chargement ;
- `Modal` — fermeture `Escape`, tailles `md/lg` ;
- `Champ`, `Input`, `Select`, `Textarea` ;
- `CarteStat`, `Badge`, `Chargeur`, `EtatVide`, `BarreRecherche`.

5.6 État global

`AuthContext` expose `utilisateur`, `chargement`, `connexion`, `deconnexion`. Au montage, si un jeton existe en session, appel `/auth/moi` pour hydrater le profil.

`ToastContext` permet d’afficher des notifications succès / erreur / info en bas à droite (auto-dismiss après 4 secondes).

5.7 Layout responsive

`Layout.tsx` combine :

- barre latérale fixe sur grand écran (navigation, profil, déconnexion);
- menu hamburger et tiroir sur mobile;
- animation `animate-fade-in` à chaque changement de route (`key={location.pathname}`).

5.8 Build et livraison

Le `frontend/Dockerfile` est multi-stage :

1. Stage build : `node:20-alpine`, `npm install`, `npm run build` (TypeScript + Vite).
2. Stage runtime : `nginx:alpine`, copie de `dist/` et `nginx.conf`.

`nginx.conf` configure le fallback SPA (`try_files`). En Compose, un proxy `/api/` optionnel existe; en production, le navigateur appelle l’API sur un hôte distinct.

5.9 Parcours utilisateur et états d’interface

Chaque page métier suit le même patron : chargement (`Chargeur`), état vide (`EtatVide`) si la liste est vide, puis contenu principal. Les actions destructives (suppression) utilisent une confirmation implicite via bouton dédié et toast d’erreur en cas d’échec API.

TABLE 5.2 – Pages et endpoints consommés

Route	Endpoints principaux
<code>/connexion</code>	POST <code>/auth/connexion</code>
<code>/ (dashboard)</code>	GET <code>/tableau-de-bord</code>
<code>/produits</code>	GET/POST/DELETE <code>/produits</code>
<code>/categories</code>	GET/POST/DELETE <code>/categories</code>
<code>/mouvements</code>	GET/POST <code>/mouvements</code>
<code>/alertes</code>	GET <code>/produits/alertes</code>

Le hook `useFetch` encapsule le cycle *chargement* → *données* → *erreur*, ce qui évite de dupliquer les `useEffect` sur chaque page.

5.10 Tests frontend

`App.test.tsx` vérifie le rendu du titre « Connexion » avec les fournisseurs de contexte et un `MemoryRouter`. La CI exécute `npm run lint` (ESLint) et `npm test - --run` (Vitest).

Les règles ESLint (config flat dans `eslint.config.js`) couvrent les hooks React et les imports inutilisés; elles complètent TypeScript qui reste la première barrière de typage au build `tsc -b`.

Chapitre 6

Persistance et modèle de données

6.1 PostgreSQL comme source de vérité

Toutes les données métier résident dans **PostgreSQL 16**. Le moteur est choisi pour :

- les transactions ACID (mouvement + mise à jour quantité en un seul commit) ;
- les contraintes d'intégrité (unicité SKU, clés étrangères) ;
- la compatibilité entre dev (Compose), CI (service container, tests SQLite séparés) et production (StatefulSet Kubernetes).

6.2 Schéma détaillé

6.2.1 Table categorie

Listing 6.1 – Structure logique categorie

```
1 CREATE TABLE categorie (  
2     id SERIAL PRIMARY KEY,  
3     nom VARCHAR UNIQUE NOT NULL,  
4     description TEXT  
5 );
```

6.2.2 Table utilisateur

Stocke `mot_de_passe_hash` uniquement. Le champ `role` accepte `admin` ou `operateur` (contrôlé applicativement).

6.2.3 Table produit

- `reference_sku` — identifiant métier unique ;
- `quantite` — stock courant, entier ≥ 0 après mouvements valides ;
- `seuil_alerte` — déclenche l'alerte lorsque `quantite` $\leq$ `seuil_alerte`.

6.2.4 Table mouvement

Journal append-only des opérations : `type_mouvement` $\in \{\text{entree}, \text{sortie}\}$, horodatage `date_mouvement`, lien vers l'auteur (`id_utilisateur`).

6.3 Requêtes métier typiques

6.3.1 Alertes

GET /produits/alertes exécute un filtre SQL du type :

```
1 SELECT * FROM produit WHERE quantite <= seuil_alerte ORDER BY quantite
   ;
```

6.3.2 Tableau de bord

Agrégations : COUNT sur produits, catégories, mouvements ; sous-requête ou filtre pour les produits en alerte renvoyés dans le même payload JSON.

6.4 Stratégie de démarrage du schéma

Deux mécanismes coexistent :

1. **Runtime** : `create_all` dans le lifespan FastAPI crée les tables si absentes, puis `seed.py`.
2. **Alembic** : migration `001_initial` versionnée pour documentation et pipelines futurs (`alembic upgrade head`).

En production, le StatefulSet PostgreSQL monte un volume persistant (`postgres.storage: 1Gi` dans `values.yaml`) : les données survivent au redémarrage du pod sous réserve du PVC.

6.5 Données de démonstration

Le seed crée notamment :

- `admin@stock.tn / Admin123!` ;
- `opérateur@stock.tn / Operateur123!` ;
- catégories et produits avec quantités variées pour illustrer les alertes.

6.6 Flux transactionnel d'un mouvement

La figure 6.1 décrit les échanges lors d'une sortie de stock : une seule transaction SQLAlchemy regroupe l'insertion du mouvement et la mise à jour de `produit.quantite`. En cas d'erreur avant le `commit`, aucune des deux écritures n'est persistée.

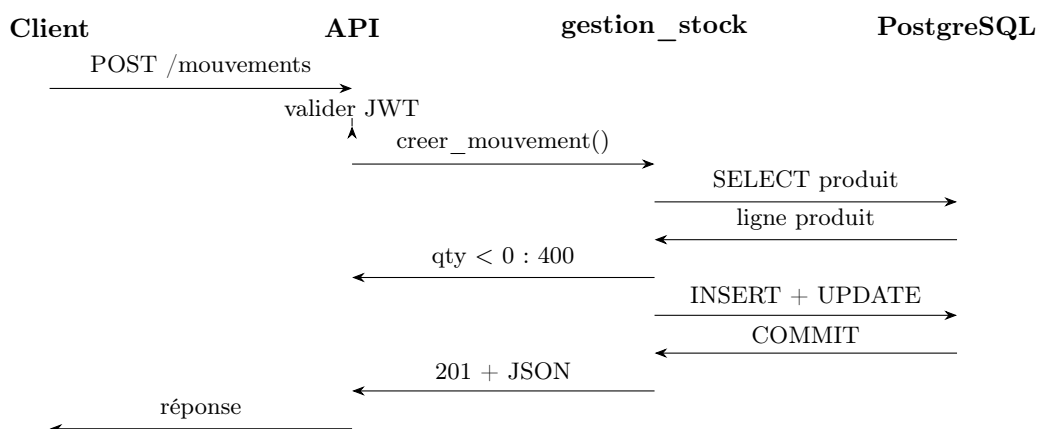


FIGURE 6.1 – Séquence simplifiée — enregistrement d'un mouvement

6.7 Relations et cardinalités

- Une **catégorie** regroupe plusieurs **produits** ; la suppression d’une catégorie référencée est bloquée par la contrainte FK (comportement REST : erreur applicative ou 409 selon le cas).
- Un **produit** possède un historique de **mouvements** ordonné par `date_mouvement` décroissant côté API.
- Chaque mouvement enregistre l’**utilisateur** auteur via `id_utilisateur`, ce qui permet une traçabilité minimale (qui a saisi l’entrée ou la sortie).

6.8 Index et performances

Le schéma initial ne déclare pas d’index secondaires explicites au-delà des clés primaires et des contraintes `UNIQUE` sur `nom` (catégorie), `email` et `reference_sku`. Pour un volume de démonstration (quelques dizaines de produits), les requêtes restent instantanées. Une montée en charge imposerait typiquement :

- un index sur `produit(id_categorie)` pour les jointures catalogue ;
- un index composite (`quantite`, `seuil_alerte`) ou une vue matérialisée pour les alertes si le filtre devient coûteux ;
- un index sur `mouvement(id_produit, date_mouvement DESC)` pour l’historique paginé.

6.9 Comparaison SQLite (tests) et PostgreSQL (runtime)

Les tests unitaires (`backend/tests/`) utilisent SQLite en mémoire : le dialecte SQLAlchemy reste identique pour les opérations CRUD simples, mais certaines subtilités (types, contraintes) diffèrent. Le runtime Compose et production cible PostgreSQL 16 ; c’est la référence pour les types `SERIAL`, `TEXT` et le comportement transactionnel réel.

6.10 Cohérence et limites

- Pas de verrouillage pessimiste explicite : en concurrence élevée, deux sorties simultanées pourraient nécessiter `SELECT FOR UPDATE` ; le périmètre actuel suppose une charge modérée.
- Pas de soft-delete : les `DELETE` sont physiques (réservés admin).
- Pas de réplication PostgreSQL en local : single instance.

Chapitre 7

Conteneurisation Docker

7.1 Rôle de Docker dans le projet

Docker matérialise l'environnement d'exécution : mêmes versions de runtime, mêmes dépendances système, isolation des services. Le fichier `docker/compose.dev.yml` couvre le développement local (dont Adminer pour visualiser PostgreSQL) ; les images construites en CI alimentent minikube et, optionnellement, GHCR pour un VPS.

7.2 Docker Compose — développement

7.2.1 Services

TABLE 7.1 – Services Compose (développement)

Service	Image	Port hôte	Rôle
postgres	postgres:16-alpine	5432	Base <code>stock_db</code>
api	build backend	8000	FastAPI
web	build frontend	3000→80	nginx + SPA
adminer	adminer:4	8080	Interface web PostgreSQL

7.2.2 Variables d'environnement API

- `DATABASE_URL=postgresql://stock:stock@postgres:5432/stock_db;`
- `SECRET_JWT, CORS_ORIGINS` incluant `http://localhost:3000`.

7.2.3 Script de démarrage

`scripts/demarrer-dev.sh` copie `.env.example` vers `.env` si nécessaire, lance `compose up -build`.

7.3 Dockerfile backend

Image basée sur `python:3.12-slim` : dépendances `pip`, `PYTHONPATH=/app`, commande `uvicorn` sur le port 8000.

7.4 Dockerfile frontend

Build multi-stage : `VITE_API_URL` figé au build. En dev Compose : `http://localhost:8000` ; pour minikube : `http://api.pfa.test` (script `demarrer-cluster-minikube.sh`).

7.5 Images et CI

Le job `docker` de la CI valide les builds. Le job `kubernetes` charge les images dans minikube. Le workflow optionnel `deploy.yml` pousse vers GHCR pour un VPS.

7.6 Réseau et volumes

Réseau `pfa` (bridge) ; volume `donnees_pg` pour persistance PostgreSQL en dev.

7.7 Bonnes pratiques

Images `alpine/slim`, secrets hors Dockerfile, builds validés en CI.

Chapitre 8

Kubernetes et Helm

8.1 Kubernetes avec minikube

8.1.1 Principe

Kubernetes orchestre des conteneurs sur un cluster : scheduling, réseau, stockage, auto-healing via sondes. Pour la **validation du PFA**, le projet utilise **minikube** (driver Docker) : cluster local reproductible, add-on ingress, déploiement Helm identique au job CI **kubernetes**.

8.1.2 Démarrage

```
1 make prerequisites-k8s
2 make cluster          # scripts/demarrer-cluster-minikube.sh
3 make demo-k8s         # cluster + smoke test in-cluster
```

Le guide docs/demo-soutenance.md détaille les entrées /etc/hosts, l'accès Grafana et minikube dashboard.

8.1.3 Production optionnelle (VPS k3s)

Un déploiement sur VPS (docs/vps-setup.md, deploy.yml manuel) reste documenté mais hors périmètre de la soutenance.

8.2 Namespaces (minikube)

Les scripts créent notamment :

- **monitoring** — kube-prometheus-stack ;
- **pfa-stock** — application métier ;
- **kubernetes-dashboard** — visualisation du cluster ;
- **pgadmin** — pgAdmin4 (visualisation BDD PostgreSQL du cluster).

8.3 Chart Helm pfa-stock

8.3.1 Values

- **values.yaml** — défauts (lint) ;
- **values-minikube.yaml** — minikube et CI : `imagePullPolicy: Never`, hôtes `pfa.test / api.pfa.test` ;
- **values-prod.yaml** — VPS optionnel (GHCR, sous-domaines publics).

8.3.2 Ressources déployées

TABLE 8.1 – Ressources du chart `pfa-stock`

Template	Kind	Détail
<code>postgres.yaml</code>	StatefulSet + Service	Volume 1Gi, image postgres :16-alpine
<code>api.yaml</code>	Deployment + Service	Probes /health, ServiceMonitor
<code>web.yaml</code>	Deployment + Service	nginx, port 80
<code>ingress.yaml</code>	Ingress (×2)	web et API sur hôtes distincts

8.3.3 Ingress et routage (minikube)

— `pfa.test` → Service `web` ;

— `api.pfa.test` → Service `api`.

Le frontend minikube est construit avec `VITE_API_URL=http://api.pfa.test`.

8.4 Observabilité et outils sur le cluster

— **kube-prometheus-stack** — Prometheus + Grafana (`grafana.test`) ;

— **Kubernetes Dashboard** — `minikube dashboard` ou `port-forward` ;

— **pgAdmin** — visualisation PostgreSQL du cluster (`pgadmin.test`) ; Adminer en Compose pour la démo Docker.

8.5 Cycle de validation

1. `make cluster` — plateforme + application sur minikube ;

2. CI — minikube éphémère, `helm upgrade`, `smoke /health` in-cluster ;

3. Optionnel — `deploy.yml` vers VPS.

Chapitre 9

Infrastructure as Code — Terraform

9.1 Objectifs de Terraform dans StockFlow

Terraform décrit l'infrastructure de façon **déclarative** et **idempotente**. Dans ce projet, le répertoire `infrastructure/terraform/` documente la **plateforme Kubernetes** (ingress, monitoring, outils) pour un déploiement VPS optionnel.

Pour la **validation du PFA**, le bootstrap local repose sur les scripts minikube (`scripts/demarrer-cluster`) qui reproduisent la même logique Helm sans `terraform apply` sur la machine de démo.

Le chart applicatif `pfa-stock` est déployé par Helm (CI minikube ou `deploy.yml` manuel), pas par Terraform.

9.2 Providers

`providers.tf` configure Kubernetes et Helm avec `config_path` pointant vers le kubeconfig de l'opérateur (`~/.kube/config` ou `KUBECONFIG`).

9.3 Variables

`variables.tf` expose notamment :

- `grafana_host`, `pgadmin_host`, `dashboard_host` ;
- `acme_email` pour Let's Encrypt (VPS) ;
- `grafana_admin_password`, `pgadmin_password` (sensibles) ;
- `postgres_password` (aligné avec le secret GitHub `POSTGRES_PASSWORD`).

9.4 Ressources par fichier

9.4.1 `kubernetes.tf`

Crée les namespaces : `ingress-nginx`, `cert-manager`, `monitoring`, `pfa-stock`, `pgadmin`, `kubernetes-dashboard`.

9.4.2 `cert-manager.tf`

Release Helm `cert-manager` et ClusterIssuer `letsencrypt-prod` (HTTP-01 via ingress nginx) — pertinent pour le VPS, pas pour minikube.

9.4.3 `helm.tf`

Values ingress VPS : `values/ingress-nginx-vps.yaml`. En local, l'addon minikube `ingress` ou les values `values-minikube.yaml` suffisent.

TABLE 9.1 – Releases Helm gérées par Terraform (VPS optionnel)

Ressource	Chart	Rôle
ingress_nginx	ingress-nginx	Entrée HTTP(S) LoadBalancer
kube_prometheus	kube-prometheus-stack	Métriques + Grafana
pgadmin	pgadmin4	UI PostgreSQL
kubernetes_dashboard	kubernetes-dashboard	Vue cluster K8s

9.4.4 grafana_dashboard.tf

ConfigMap du dashboard JSON `pfa-stock-api` pour le sidecar Grafana. Le script `minikube` applique le même manifeste via `kubectl apply`.

9.5 Exécution opérationnelle

9.5.1 Validation (CI et local)

```
1 cd infrastructure/terraform
2 terraform init -backend=false
3 terraform fmt -check
4 terraform validate
```

Le job `terraform` de la CI exécute ces commandes. Aucun `apply` sur les runners GitHub.

9.5.2 Apply production (optionnel)

```
1 terraform init
2 terraform apply -var="grafana_admin_password=..." -var="
  pgadmin_password=..."
```

À exécuter depuis une machine dont le `KUBECONFIG` cible un VPS k3s (voir `docs/vps-setup.md`). Le state reste local (`terraform.tfstate`, gitignoré).

9.6 Terraform vs scripts minikube

TABLE 9.2 – Terraform vs bootstrap minikube

Composant	Terraform (VPS)	Script minikube
Ingress	ingress-nginx Helm	addon <code>ingress</code>
Monitoring	kube-prometheus-stack	<code>kube-prometheus-values-minikube.yaml</code>
Dashboard K8s	Helm release	tarball GitHub + values minikube
App	—	<code>helm upgrade pfa-stock</code>
Validation CI	<code>validate</code>	job <code>kubernetes</code>

Voir `infrastructure/terraform/README.md` et `docs/demo-soutenance.md`.

Chapitre 10

Observabilité — Prometheus et Grafana

10.1 Objectifs

L'observabilité vise à répondre, en exploitation :

- l'API reçoit-elle du trafic ? à quel rythme ?
- quelle est la latence (p50, p95, p99) ?
- y a-t-il des erreurs 5xx ?
- quelles routes sont les plus sollicitées ?

StockFlow adopte la stack **Prometheus** (collecte) + **Grafana** (visualisation), standard dans l'écosystème Kubernetes.

10.2 Instrumentation applicative

Le package `prometheus-fastapi-instrumentator` expose automatiquement sur `GET /metrics` :

- compteur `http_requests_total` (labels `method`, `handler`, `status`) ;
- histogrammes de durée de requête ;
- métriques en vol (`inprogress`).

Aucune modification manuelle par route n'est nécessaire pour les métriques HTTP de base.

10.3 kube-prometheus-stack

10.3.1 Démo locale (minikube)

Le script `demarrer-cluster-minikube.sh` installe le chart `kube-prometheus-stack` avec `monitoring/kube-prometheus-values-minikube.yaml` :

- ressources réduites (Alertmanager désactivé, rétention courte) ;
- Grafana accessible via ingress `grafana.test` ou `minikube service` ;
- sidecar dashboards — surveille les ConfigMaps `grafana_dashboard`: "1".

10.3.2 Production optionnelle (VPS)

Terraform peut installer le même chart avec `monitoring/kube-prometheus-values.yaml` et TLS sur `grafana.pfa.elyes.dev`.

10.4 ServiceMonitor

Le CRD `ServiceMonitor` (Prometheus Operator) décrit comment scraper une cible. Le chart `pfa-stock` crée :

Listing 10.1 – Extrait `ServiceMonitor` (conceptuel)

```

1 spec:
2   selector:
3     matchLabels:
4       app: api
5   endpoints:
6     - port: http
7       path: /metrics
8       interval: 30s

```

Le label `release: kube-prometheus-stack` associe ce monitor à l'instance Prometheus déployée par le chart homonyme.

10.5 Flux de métriques

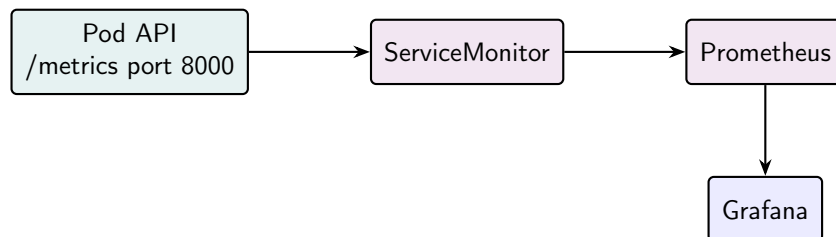


FIGURE 10.1 – Chemin des métriques

10.6 Dashboard Grafana

Le fichier `monitoring/dashboards/pfa-stock-api.json` définit le tableau de bord *PFA Stock — API FastAPI* avec panneaux :

- débit de requêtes ;
- erreurs 5xx ;
- latences p50 / p95 / p99 ;
- répartition par route et par code HTTP ;
- requêtes en cours.

Le manifeste `ConfigMap` est appliqué par le script `minikube` (ou `Terraform` sur VPS) ; le sidebar Grafana charge le dashboard — versionné comme le code.

10.7 Accès opérationnel

10.7.1 Démo soutenance (minikube)

- Grafana : `http://grafana.test` (entrée `/etc/hosts` vers minikube ip) ou port-forward affiché par `make cluster` ;
- identifiants : `admin` / mot de passe défini dans `kube-prometheus-values-minikube.yaml`.

10.7.2 Production optionnelle

Grafana sur `https://grafana.pfa.elyes.dev` après `terraform apply` sur VPS.

10.8 Mode développement Compose

En `make dev`, Prometheus/Grafana ne sont pas déployés. Les métriques restent accessibles sur `http://localhost:8000/metrics`. Adminer (`http://localhost:8080`) couvre la visualisation PostgreSQL sur ce chemin.

10.9 Alerting

Alertmanager est inclus dans kube-prometheus-stack mais désactivé en minikube pour économiser la RAM. Les règles d'alerte custom (stock critique, API down prolongé) ne sont pas configurées — extension naturelle pour une phase ultérieure.

Chapitre 11

Intégration continue et déploiement — GitHub Actions

11.1 Principes

Deux workflows :

- `ci.yml` — validation complète (tests, build, Terraform, Helm, **minikube**) ;
- `deploy.yml` — déploiement VPS **optionnel** (`workflow_dispatch` uniquement).

11.2 Déclencheurs

- CI : push sur `main/develop`, `pull_request` vers `main` ;
- CD : `workflow_dispatch` seulement (pas de déploiement auto sur push).

11.3 Pipeline CI

Six jobs : `backend`, `frontend`, `docker`, `terraform`, `helm`, `kubernetes`.

Le job `kubernetes` :

1. démarre `minikube` (driver Docker) ;
2. build et `minikube image load` des images API/Web ;
3. active l'addon `ingress` ;
4. `helm upgrade -install pfa-stock` avec `values-minikube.yaml` ;
5. smoke test in-cluster : `curl http://api:8000/health`.

11.4 Pipeline CD optionnel (`deploy.yml`)

Déclenchement manuel : build GHCR, `helm upgrade` sur VPS (`values-prod.yaml`), smoke in-cluster et HTTPS public (`continue-on-error`).

11.5 Comparaison CI vs local

11.6 Gouvernance

Les PR doivent passer la CI avant merge. La validation PFA repose sur la CI et la démo locale (`docs/demo-soutenance.md`), sans serveur de production obligatoire.

TABLE 11.1 – CI vs commandes locales

Vérification	CI	make
pytest / vitest	Oui	<code>make test</code>
docker build	Oui	<code>make dev</code>
terraform / helm	Oui	—
minikube deploy	Oui	<code>make cluster</code>
Grafana / dashboard	Démo locale	<code>make cluster</code>

Chapitre 12

Sécurité, tests et qualité

12.1 Modèle de menace (périmètre local)

Le déploiement couvre le **développement local** et la **production sur VPS dédié** (TLS, secrets GitHub). Les menaces principales sont : accès non autorisé à l'API, fuite de secrets dans Git, injection SQL (mitigée par ORM), et configuration CORS trop permissive.

12.2 Authentification et autorisation

- Mots de passe hashés bcrypt — pas de stockage clair ;
- JWT signé HS256 — secret `SECRET_JWT` injecté par variable d'environnement ou Secret K8s ;
- Durée de vie configurable (`duree_token_minutes`) ;
- Contrôles de rôle sur DELETE (vérification `role == admin`).

12.3 Secrets et configuration

TABLE 12.1 – Gestion des secrets

Secret	Stockage
<code>SECRET_JWT</code>	Secret K8s / <code>.env</code> (gitignoré)
<code>DATABASE_URL</code>	Secret K8s (mot de passe PG inclus)
Mots de passe démo	Seed — à changer en prod
Grafana admin	<code>TF_VAR_grafana_admin_password</code>

`.env.example` documente les clés sans valeurs sensibles réelles. `.gitignore` exclut `.env`, `.kube/`, `*.tfstate`.

12.4 CORS

`CORS_ORIGINS` liste les origines autorisées (dev : `localhost:3000` ; production : `https://pfa.elyes.dev`).

12.5 Réseau Kubernetes

Les Services clusterIP ne sont pas exposés sauf via Ingress (80/443). L'API n'écoute pas sur les ports hôte 8000, 6001 ou 6002.

12.6 Tests automatisés

12.6.1 Backend (pytest)

Quatre tests couvrent :

- connexion + liste produits;
- création produit;
- entrée de stock;
- sortie refusée si stock insuffisant (`stock_insuffisant`).

Fixtures : client FastAPI, jeton admin, base SQLite mémoire, override de `obtenir_session`.

Le fichier `conftest.py` isole chaque test :

Listing 12.1 – Stratégie de test (extrait `conftest`)

```
1 os.environ["MODE_TEST"] = "1"
2 # SQLite :memory: + create_all avant chaque fonction de test
3 @pytest.fixture
4 def client():
5     with TestClient(application) as c:
6         yield c
```

Cette approche garantit un état vierge par test sans dépendre d'un Postgres déjà peuplé par le seed de production.

12.6.2 Frontend (Vitest)

Test de rendu de la page connexion avec providers Auth/Toast et MemoryRouter.

12.6.3 Smoke tests (make verify)

`scripts/verifier.sh` enchaîne :

- `curl` sur `/health`, login, liste produits, page web (stack dev locale).

12.7 Analyse statique

- **ruff** — backend Python (CI + `make lint`);
- **eslint** — frontend TypeScript/React (CI).

12.8 Pistes de durcissement production

- TLS terminé par `cert-manager` + Let's Encrypt;
- rotation des secrets via External Secrets Operator;
- politiques NetworkPolicy limitant le trafic inter-pods;
- rate limiting sur l'Ingress;
- audit des dépendances (`pip audit`, `npm audit`).

Chapitre 13

Exploitation et scripts d'automatisation

13.1 Makefile comme point d'entrée

TABLE 13.1 – Commandes Make principales

Cible	Action
prerequis	Vérifie Docker et Compose
prerequis-k8s	Vérifie minikube, kubectl, helm
dev	Lance Compose dev (app + Adminer)
cluster	Démarre minikube + plateforme + app
cluster-down	Arrête ou supprime le cluster minikube
demo-k8s	Cluster + smoke test in-cluster
demo	dev + tests + liens
test / verify / lint	Qualité et smoke tests
links	URLs locales (Compose et minikube)
destroy	Arrêt stack dev

13.2 Scripts shell

TABLE 13.2 – Scripts du répertoire `scripts/`

Script	Rôle
demarrer-dev.sh	Compose dev
demarrer-cluster-minikube.sh	Bootstrap minikube complet
arreter-cluster-minikube.sh	Arrêt cluster minikube
verifier-cluster-minikube.sh	Smoke test K8s
verifier-prerequis-k8s.sh	Prérequis minikube
demarrer-tout.sh	Demo Compose complète
verifier.sh	Tests smoke HTTP
afficher-acces.sh	URLs et identifiants
arreter-tout.sh	Arrêt Compose

13.3 Guide soutien

`docs/demo-soutenance.md` décrit l'ordre de démo, les prérequis (8 Go RAM), les captures d'écran attendues et la checklist GitHub Actions (6 jobs verts).

13.4 URLs

13.4.1 Développement local (Compose)

- UI : `http://localhost:3000`
- API : `http://localhost:8000/docs`
- Adminer : `http://localhost:8080` (serveur `postgres`, user `stock`)

13.4.2 Validation Kubernetes (minikube)

- Entrées `/etc/hosts` : `$(minikube ip) pfa.test api.pfa.test grafana.test pgadmin.test`
- Application : `http://pfa.test`
 - API : `http://api.pfa.test/docs`
 - Grafana : `http://grafana.test`
 - pgAdmin : `http://pgadmin.test`
 - Dashboard K8s : `minikube dashboard` ou `port-forward` (script)
- Ne pas lancer `Compose app` et `minikube app` simultanément (deux instances PostgreSQL).

13.4.3 Production optionnelle (VPS k3s)

- Application : `https://pfa.elyes.dev`
 - API : `https://api.pfa.elyes.dev`
 - Grafana : `https://grafana.pfa.elyes.dev`
- Voir `docs/vps-setup.md` et `deploy.yml` (déclenchement manuel).

13.5 Dépannage

minikube OOM Réduire `-memory` ou désactiver `pgAdmin` cluster (`WITH_PGADMIN=false`).

Ingress 404 Vérifier `/etc/hosts` et `kubectl get ingress -A`.

ImagePullBackOff minikube Images locales : `imagePullPolicy: Never` dans `values-minikube.yaml`.

ImagePullBackOff VPS Secret `ghcr-secret` ou visibilité packages GHCR.

13.6 Logs

```
1 kubectl logs -n pfa-stock deployment/api -f
2 docker compose -f docker/compose.dev.yml logs api
3 minikube logs
```

Chapitre 14

Conclusion et perspectives

14.1 Bilan

Le projet **StockFlow** démontre qu’une application métier de gestion de stock peut être livrée avec une chaîne DevOps complète : développement local Docker Compose (avec Adminer), validation Kubernetes sur minikube, observabilité Prometheus/Grafana, visualisation du cluster et CI/CD GitHub Actions — le tout **sans serveur de production obligatoire**.

14.2 Objectifs atteints

- Fonctionnalités métier : catalogue, mouvements, alertes, tableau de bord ;
- Développement local via **make dev** (Adminer pour PostgreSQL) ;
- Validation K8s via **make cluster** (minikube, Helm, Grafana, dashboard) ;
- CI avec six jobs dont déploiement éphémère minikube ;
- Infrastructure as Code (Terraform valide en CI ; apply VPS optionnel) ;
- Déploiement production documenté (**deploy.yml** manuel, **docs/vps-setup.md**).

14.3 Limites connues

- Cluster mono-nœud (minikube ou k3s) ;
- State Terraform local ;
- Tests CI encore majoritairement SQLite ;
- Monitoring complet (kube-prometheus) non exécuté dans le job CI minikube (RAM) — preuve en démo locale ;
- Identifiants de démo à renforcer en production réelle.

14.4 Perspectives

1. Backend Terraform distant (S3 + DynamoDB) ;
2. Haute disponibilité multi-nœuds ;
3. GitOps (Flux / Argo CD) ;
4. Tests e2e post-déploiement ;
5. Notifications sur alertes stock ;
6. Déploiement VPS stabilisé si ressources matérielles suffisantes.

14.5 Mot de fin

StockFlow illustre l'unification code applicatif et infrastructure dans un même dépôt, avec deux chemins de validation complémentaires — Docker Compose pour le développement, minikube pour l'orchestration — et une CI qui garantit la reproductibilité du déploiement Helm.

Annexe A

Annexe A — Référence API REST

Préfixe commun : `/api/v1`. Authentification : `Authorization: Bearer <jeton>` sauf connexion.

A.1 Auth

TABLE A.1 – Endpoints d’authentification

Méthode	Chemin	Corps / réponse
POST	<code>/auth/connexion</code>	JSON <code>{email, mot_de_passe}</code> → <code>{jeton_acces}</code>
GET	<code>/auth/moi</code>	Profil utilisateur courant

A.2 Catégories

CRUD standard : `GET/POST /categories`, `GET/PUT/DELETE /categories/{id}`. `DELETE` réservé admin.

A.3 Produits

`GET /produits`, `GET /produits/alertes`, `POST /produits`, `GET/PUT/DELETE /produits/{id}`.
Champs création : `nom`, `reference_sku`, `quantite`, `seuil_alerte`, `id_categorie`.

A.4 Mouvements

`GET /mouvements`, `POST /mouvements` avec `type_mouvement` (`entree|sortie`), `quantite`, `id_produit`, motif optionnel.
Erreur métier : `400 stock_insuffisant`.

A.5 Tableau de bord

`GET /tableau-de-bord` — JSON avec compteurs et liste `produits_alerte`.

A.6 Exports

A.7 Santé et métriques

GET /health — {"statut":"ok"} (ou équivalent).
GET /metrics — format Prometheus texte.

A.8 Exemples de payloads JSON

A.8.1 Connexion

Requête :

```
1 POST /api/v1/auth/connexion
2 {"email": "admin@stock.tn", "mot_de_passe": "Admin123!"}
```

Réponse 200 :

```
1 {"jeton_acces": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."}
```

A.8.2 Création de produit

```
1 POST /api/v1/produits
2 {
3   "nom": "Clavier mecanique",
4   "reference_sku": "KB-MECH-01",
5   "quantite": 25,
6   "seuil_alerte": 5,
7   "id_categorie": 1
8 }
```

A.8.3 Mouvement de sortie

```
1 POST /api/v1/mouvements
2 {
3   "type_mouvement": "sortie",
4   "quantite": 3,
5   "id_produit": 2,
6   "motif": "Vente comptoir"
7 }
```

A.8.4 Tableau de bord (extrait de réponse)

```
1 {
2   "nombre_produits": 12,
3   "nombre_categories": 4,
4   "nombre_mouvements": 28,
5   "nombre_alertes": 2,
6   "produits_alerte": [ ... ]
7 }
```

TABLE A.2 – Erreurs API typiques

Code	detail	Contexte
401	jeton_invalide / non authentifié	Jeton absent, expiré ou signature incorrecte
403	accès refusé	Rôle insuffisant (ex. export admin)
404	produit_introuvable	ID produit inexistant
400	stock_insuffisant	Sortie dépassant le stock disponible
422	validation Pydantic	Corps JSON mal formé ou champs manquants

A.9 Codes d'erreur HTTP courants

A.10 Exemples cURL (mode Compose)

Avec l'API sur localhost:8000 et un jeton dans la variable \$TOKEN :

```
1 curl -s http://localhost:8000/health
2 curl -s -H "Authorization: Bearer $TOKEN" \
3   http://localhost:8000/api/v1/tableau-de-bord | jq
4 curl -s -X POST -H "Authorization: Bearer $TOKEN" \
5   -H "Content-Type: application/json" \
6   -d '{"type_mouvement": "entree", "quantite": 10, "id_produit": 1}' \
7   http://localhost:8000/api/v1/mouvements
```

En production : <https://api.pfa.elyes.dev/api/v1/...>

Annexe B

Annexe B — Configuration et versions

B.1 Variables d’environnement (`.env.example`)

TABLE B.1 – Variables d’environnement principales

Variable	Description
DATABASE_URL	URL SQLAlchemy PostgreSQL
SECRET_JWT	Clé signature JWT
CORS_ORIGINS	Liste séparée par virgules
VITE_API_URL	URL API pour build frontend
POSTGRES_*	Paramètres PostgreSQL (Compose/Helm)

B.2 Secrets GitHub Actions (production optionnelle)

TABLE B.2 – Secrets repository pour `deploy.yml` (déclenchement manuel)

Secret	Usage
KUBECONFIG	Accès k3s (base64)
SECRET_JWT	JWT production
POSTGRES_PASSWORD	PostgreSQL
GHCR_PAT	Pull images privées

B.3 Versions d’outils (référence)

B.4 Fichiers de référence rapide

- Guide soutenance : `docs/demo-soutenance.md`
- CI : `.github/workflows/ci.yml` (6 jobs, dont `kubernetes`)
- CD optionnel : `.github/workflows/deploy.yml` (`workflow_dispatch`)
- Compose dev : `docker/compose.dev.yml` (Adminer :8080)
- Chart minikube : `helm/pfa-stock/values-minikube.yaml`
- Chart prod : `helm/pfa-stock/values-prod.yaml`
- Bootstrap K8s local : `scripts/demarrer-cluster-minikube.sh`
- Terraform : `infrastructure/terraform/README.md`
- Guide VPS (annexe) : `docs/vps-setup.md`

TABLE B.3 – Versions d’outils (référence)

Composant	Version
Python	3.12
Node.js	20
PostgreSQL	16
minikube	latest (driver Docker)
k3s	latest (VPS optionnel)
Helm chart ingress-nginx	4.11.3
cert-manager	v1.16.2
kube-prometheus-stack	65.5.0